



PAW HEALTH

REPORT

Android studio

Mobile programming

The pet is human friend. So take care of it.

PAWhealth Application Report

1. Introduction :

The PAWhealth application was developed as part of a university project to address the needs of pet owners. It centralizes all essential information about their pets, making it easier to manage their well-being.

2. Application Objective :

PAW health aims to help pet owners manage and centralize all the information related to their pets. Users can track their pet's favorite food, remember the last bath date, keep records of vaccinations, and take specific notes.

3. Target Audience :

The application is designed for anyone who owns a pet, regardless of age, gender, or profession. It aims to simplify daily life for pet owners by providing a straightforward and efficient platform to manage their pets' needs.

4. Application Features :

- * Food Management: Allows users to note their pet's favorite food.
- * Care Tracking: Reminds users of the last bath date.
- * Vaccinations: Records vaccinations to keep track of them easily.
- * Personal Notes: Section for adding specific notes or important information about the pet.
- * WE TRY TO IT: Profile Photo: Option to add a profile photo for each pet.

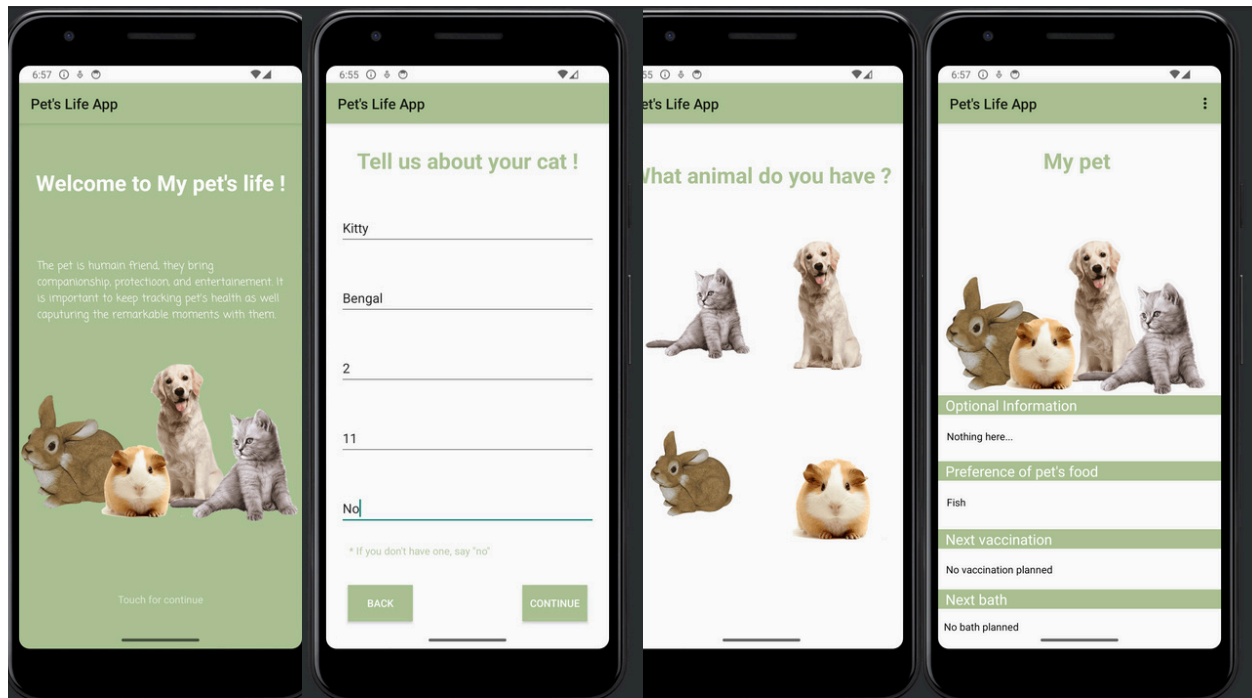
5. Technologies Used :

- * Programming Language: Java
- * Development Environment: Android Studio
- * Database: SQLite for local data storage

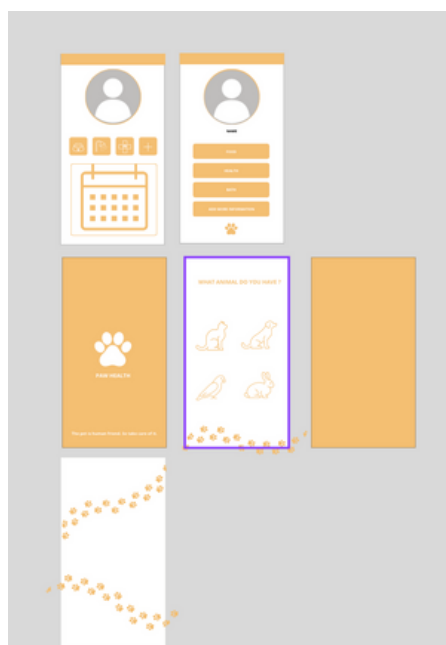
6. Development Process

We adopted an iterative approach for the application development. The main steps were:

- Phase 1: Initial Version
 - Initial design of the application.
 - Development of the first version with basic features to test information management.



- Phase 2: Design Improvement
 - Redesign of the user interface to enhance user experience (UX).
 - Addition of new features and optimization of existing ones.

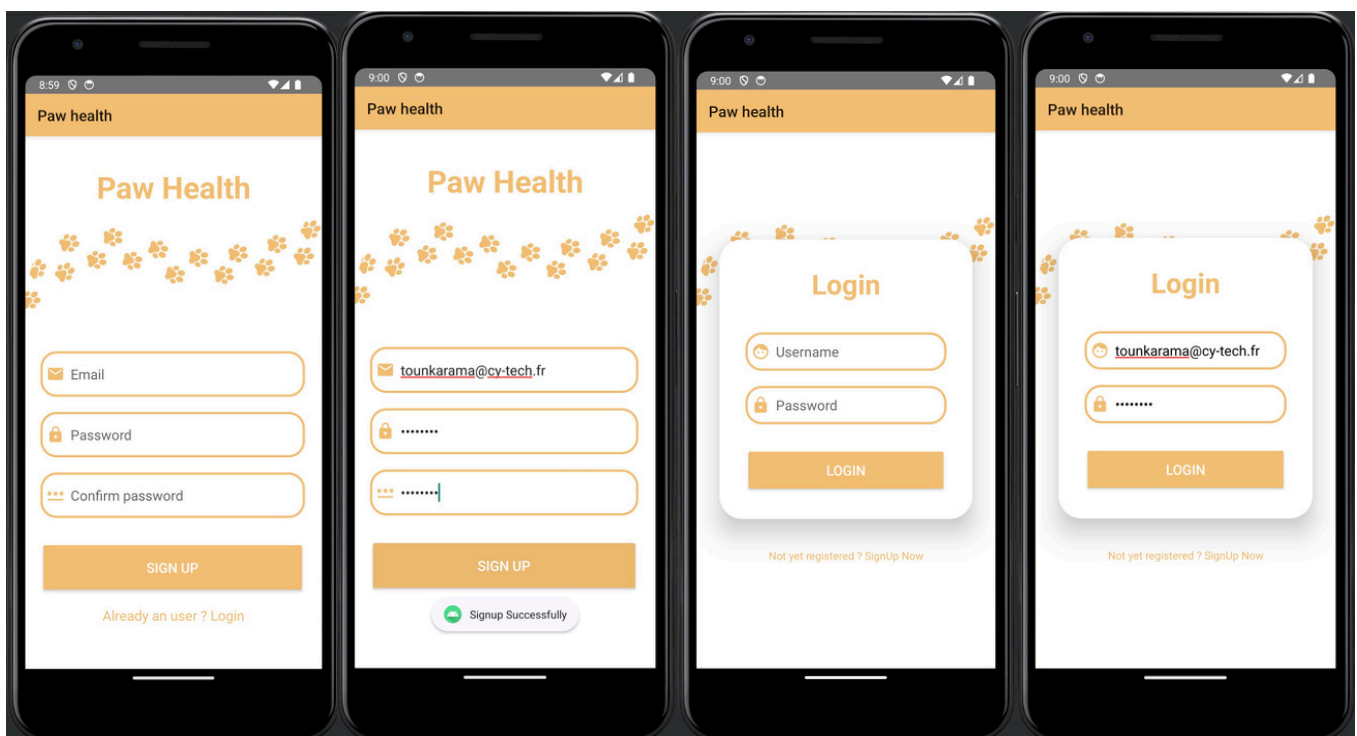


- Phase 3: Login System Implementation
 - Development of a login system to secure user data and enable personal information recovery.

7. Development of the Login System

The development of the login system for PAWhealth involved several crucial steps:

- Sign Up:
 - Users can create an account by providing an email address and a password.
- Login:
 - Users can log in with their credentials to access their information.
 - Once logged in, user data is retrieved from the SQLite database and displayed in the application.
- Profile Management:
 - During the first login, users must enter their pet's information.
 - Information is stored locally in SQLite for quick access.
 - Pet profile information is also saved in SharedPreferences for quick retrieval and a smooth user experience.



8. Main Class Structure

(in the annexes part you will find all the class)

9. Design and User Interface

For the design, we followed Material Design guidelines to ensure a consistent and intuitive user interface. User tests were conducted to gather feedback and improve the application's ergonomics.

10. Technical Challenges Encountered

- * We found a difficulty to communicate with the data base and in particular a table
- * We try do to give the possibility to the user to changer his profile picture, but we steel work on it. It's little difficult because we want to include it on the menu
- * The SharedPreferences was difficult to manipulate

11. Future Improvements

- * Multiple Pet Profiles: Add a feature allowing users to manage multiple pet profiles, making it easier for those with more than one pet to switch between profiles.
- * Push Notifications: Add notifications to remind users of important dates like vaccinations and baths.

12. References

This is all the video that help us to solution all ours problems:

- * <https://youtu.be/9t8VWWebRFM?si=WNiLEQff7TLIVsFe>
- * https://youtu.be/qCcSE1_yTCI?si=Dingg12XDMD05Out
- * <https://youtu.be/qHZcHsnoyCs?si=4cuAYAK6k5jCAbDO>
- * https://youtu.be/J-CP7g_Gwpl?si=93s560-_mBYjzmdX

This all the video with options that we see and we want to work on :

- * <https://youtu.be/-PMxYVNGl1c?si=UOCRNVzYr-bZ7ESI>
- * <https://www.youtube.com/watch?v=v6YvUxpgSYQ>
- * <https://www.youtube.com/watch?v=bou80uTGxs4>

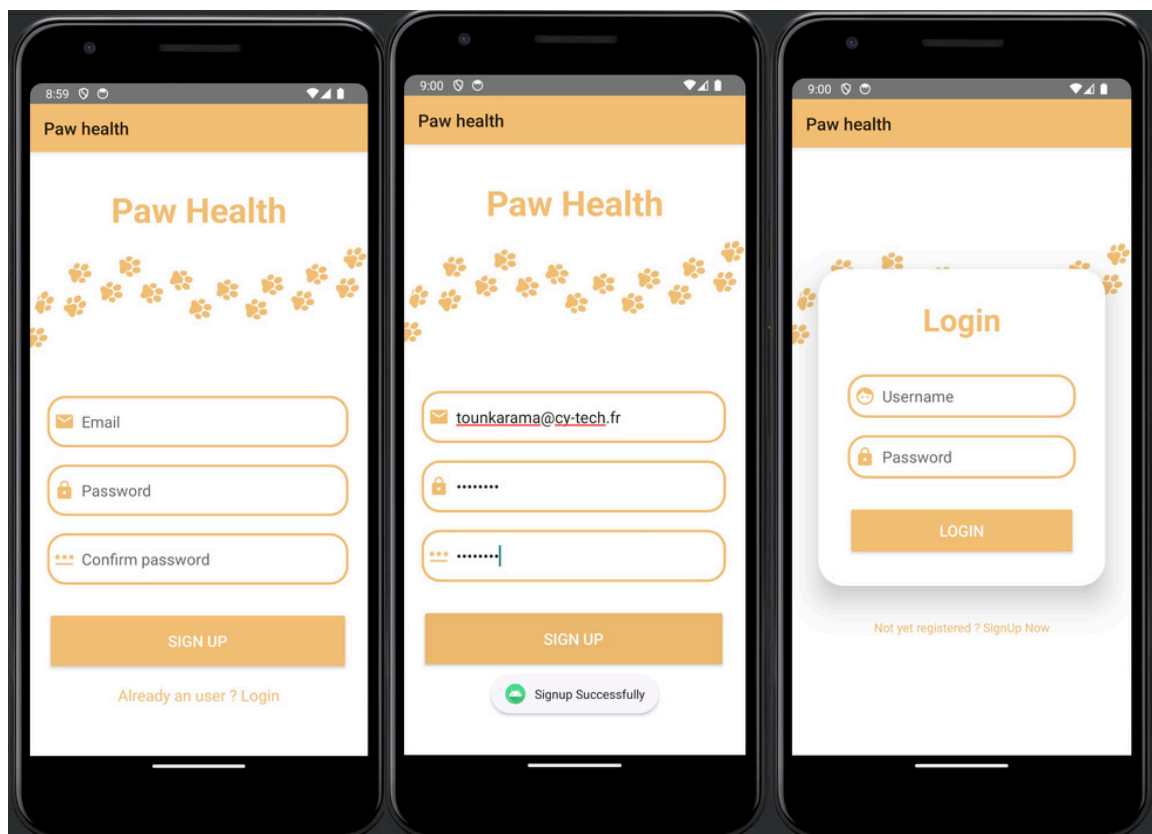
13. Project Evaluation

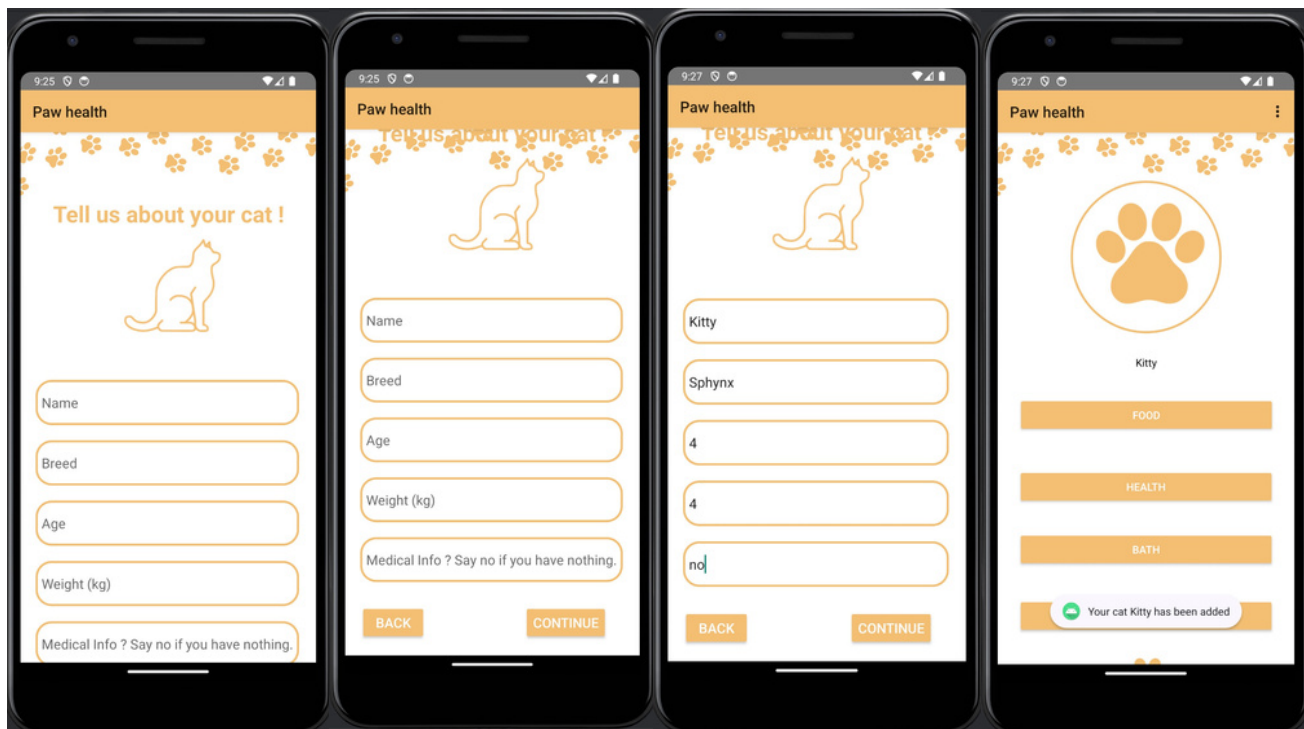
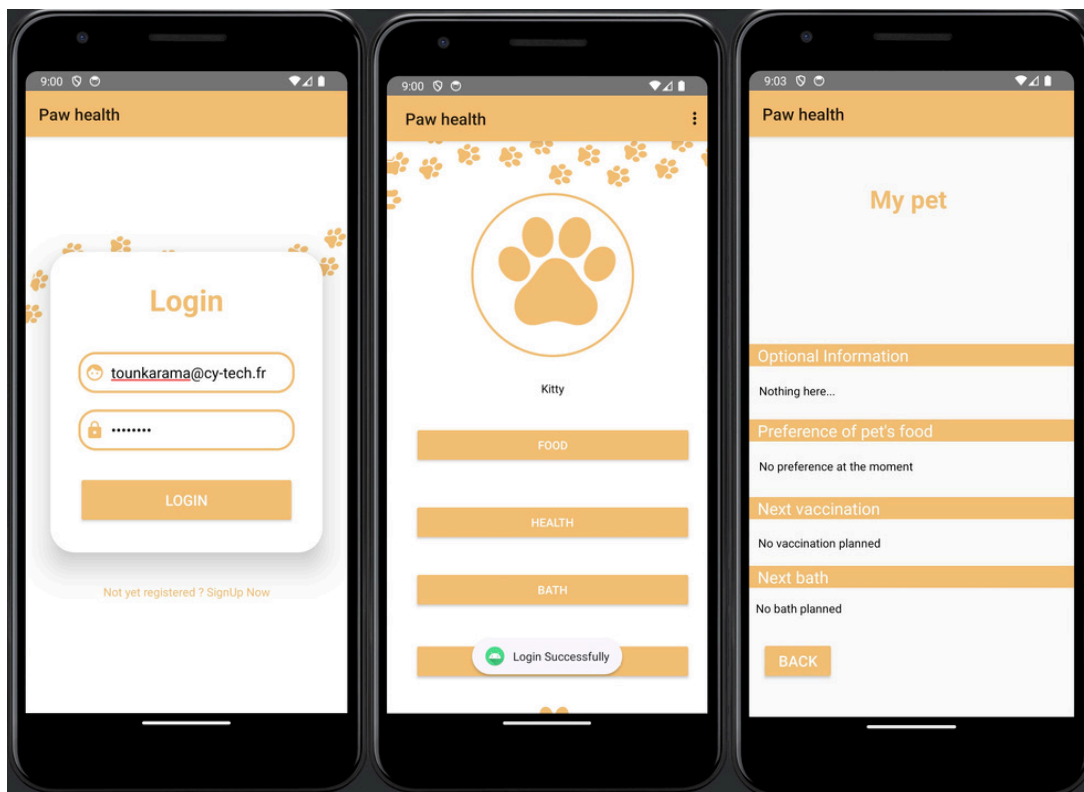
We found this project very interesting as it required us to reassess how to manage the project effectively. We learned the importance of detailing what we wanted to achieve to better manage the execution and find everything we needed. This project also allowed us to strengthen our skills in mobile development and teamwork.

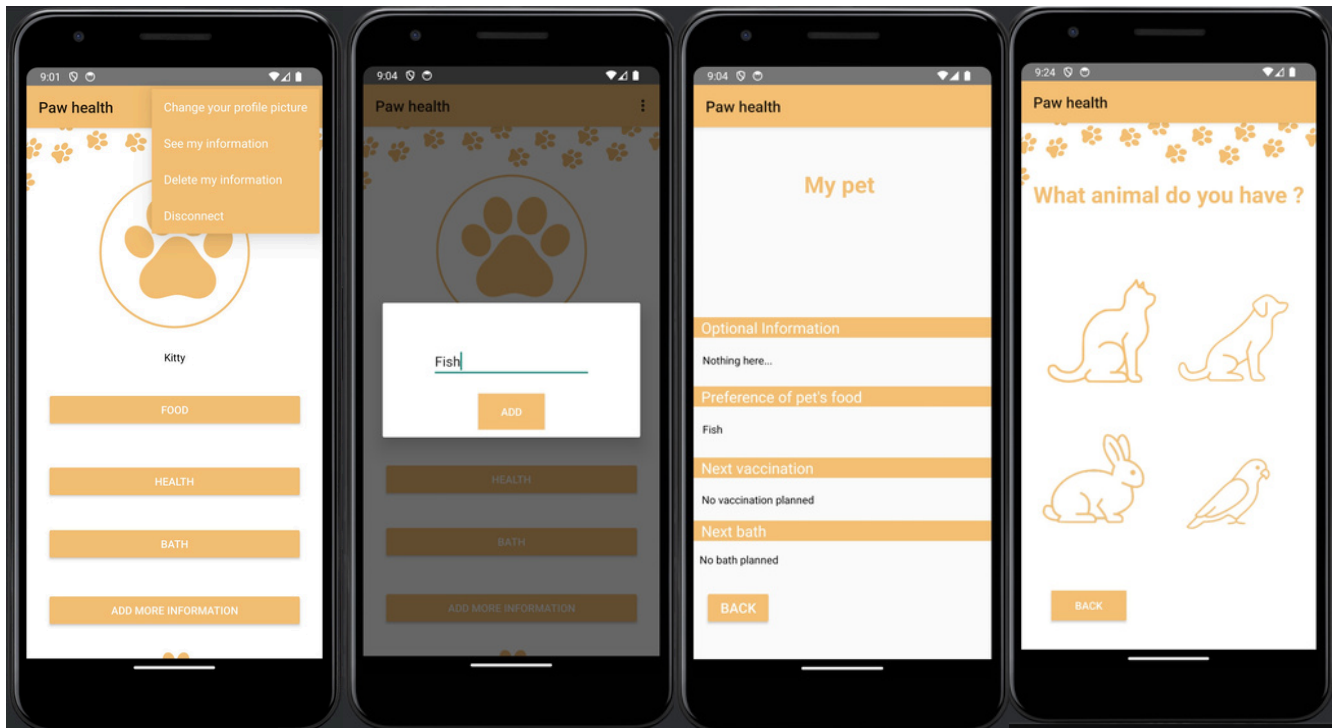
Mariam: LogIN, SQLite, Menu, SharedPreferences, HomePage

Lyna: Interface, profile picture, the questionnaire, HomePage

14.ANNEXES







MAIN(Changes activity after 4 seconds)

```
package com.example.pawhealth;

import ...

public class MainActivity extends AppCompatActivity {
    1 usage
    private static int TIME_OUT = 4000;
    2 usages
    Timer timer;
    no usages
    DBHelper db;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        timer = new Timer();
        timer.schedule(new TimerTask() {
            @Override
            public void run() {
                Intent intent = new Intent( packageContext: MainActivity.this, SignUpActivity.class);
                startActivity(intent);
                finish();
            }
        }, TIME_OUT);
    }
}
```

SIGNUP (tests if the user already exists)

```
btnSignup.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        String email = signupEmail.getText().toString();  
        String password = signupPassword.getText().toString();  
        String confirmPassword = signupConfirm.getText().toString();  
  
        if(email.equals("") || password.equals("") || confirmPassword.equals("")){  
            Toast.makeText(context: SignUpActivity.this, text: "All fields are mandatory", Toast.LENGTH_SHORT).show();  
        } else {  
            if (password.equals(confirmPassword)) {  
                // Boolean checkUserEmail = db.checkEmail(email);  
                // if (checkUserEmail == false) {  
                    Boolean insert = db.insertData(email, password);  
                    if (insert == true) {  
                        Toast.makeText(context: SignUpActivity.this, text: "Signup Successfully", Toast.LENGTH_SHORT).show();  
                        Intent intent = new Intent(getApplicationContext(), LoginActivity.class);  
                        startActivity(intent);  
                    } else {  
                        Toast.makeText(context: SignUpActivity.this, text: "Signup Failed", Toast.LENGTH_SHORT).show();  
                    }  
                } //} else {  
                //    Toast.makeText(SignUpActivity.this, "User already exists. Please login", Toast.LENGTH_SHORT).show();  
                //}  
            } else {  
                Toast.makeText(context: SignUpActivity.this, text: "Invalid Password", Toast.LENGTH_SHORT).show();  
            }  
        }  
    }  
}
```

Login (tests if the user have an account and if the correct information)

```
btnLogin.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        String email = username.getText().toString();  
        String passwordTXT = password.getText().toString();  
        Cursor cursor = petInfo.getdata();  
  
        if(email.equals("") || passwordTXT.equals("")){  
            Toast.makeText(context: LoginActivity.this, text: "All fields are mandatory", Toast.LENGTH_SHORT).show();  
        }else {  
            Cursor res= db.checkEmailPassword(email, passwordTXT);  
  
            if(res.getCount()== 1){  
                Toast.makeText(context: LoginActivity.this, text: "Login Successfully", Toast.LENGTH_SHORT).show();  
                if(cursor.getCount() == 0) {  
                    Intent intent = new Intent(packageContext: LoginActivity.this, PetsChoose.class);  
                    startActivity(intent);  
                }else {  
                    Intent intent = new Intent(packageContext: LoginActivity.this, HomePage.class);  
                    startActivity(intent);  
                }  
            } else{  
                Toast.makeText(context: LoginActivity.this, text: "Invalid Credentials", Toast.LENGTH_SHORT).show();  
            }  
        }  
    }  
}
```

SQLite for user information

```
10 // usage
11 public class DBHelper extends SQLiteOpenHelper {
12
13     3 usages
14     public DBHelper(Context context){
15         super(context, "Signup.db", factory: null, version: 1);
16     }
17
18     @Override
19     public void onCreate(SQLiteDatabase db) {
20         db.execSQL("CREATE TABLE allusers(email TEXT PRIMARY KEY, password TEXT);");
21     }
22
23     10 usages
24     @Override
25     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
26         db.execSQL("DROP TABLE IF EXISTS allusers");
27     }
28
29     1 usage
30     public Boolean insertData(String email, String password){
31         SQLiteDatabase db = this.getWritableDatabase();
32         ContentValues contentValues = new ContentValues();
33         contentValues.put("email", email);
34         contentValues.put("password", password);
35         long result = db.insert("allusers", null, contentValues);
36         if (result == -1) {
37             return false;
38         } else {
39             return true;
40         }
41     }
42
43     no usages
44     public Boolean checkEmail(String email){
45         SQLiteDatabase db = this.getWritableDatabase();
46         Cursor cursor = db.rawQuery("SELECT * FROM users where email = ?", new String[]{email});
47         if(cursor.getCount() > 0) {
48             return true;
49         } else {
50             return false;
51         }
52     }
53
54     1 usage
55     public Cursor checkEmailAndPassword(String email, String password){
56         SQLiteDatabase db = this.getWritableDatabase();
57         //Cursor cursor = db.rawQuery("SELECT * FROM users where email = ? and password = ?", new String[]{email, password});
58         return db.rawQuery("SELECT * FROM allusers where email = ? AND password = ?", new String[]{email, password}, null, null, null);
59     }
60
61     1 usage
62     void deleteAllData(){
63         SQLiteDatabase db = this.getWritableDatabase();
64         db.execSQL("DELETE FROM allusers");
65     }
66
67     no usages
68     public Cursor getdata(){
69         SQLiteDatabase db = this.getWritableDatabase();
70         Cursor cursor = db.rawQuery("SELECT * FROM PetInfo", selectionArgs: null);
71         return cursor;
72     }
73 }
```

SQLite for pet info

```
Paw health - Version control -
@ HomePage.java @ MainActivity.java @ OptionInfo.java @ LoginActivity.java @ SignUpActivity.java @ DBHelper.java @ PetInfo.java
15 public PetInfo(Context context) { super(context, name, "PetInfo.db", factory: null, version: 1); }
16
17 @Override
18 public void onCreate(SQLiteDatabase db) {
19     db.execSQL("CREATE TABLE IF NOT EXISTS PetInfo(name TEXT primary key, breed TEXT, age INTEGER, weight INTEGER, medicalInfo TEXT);");
20 }
21
22
23
24
25 10 usages
26 public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
27     db.execSQL("DROP TABLE IF EXISTS PetInfo");
28 }
29
30 4 usages
31 public Boolean insertPet(String name, String breed, int age, int weight, String medicalInfo) {
32     SQLiteDatabase db = this.getWritableDatabase();
33     ContentValues contentValues = new ContentValues();
34     contentValues.put("name", name);
35     contentValues.put("breed", breed);
36     contentValues.put("age", age);
37     contentValues.put("weight", weight);
38     contentValues.put("medicalInfo", medicalInfo);
39     long result = db.insert(tableName, null, contentValues);
40     if(result == -1) {
41         return false;
42     } else {
43         return true;
44     }
45 }
46
47 10 usages
48 public Boolean updatePet(String name, String breed, int age, int weight, String medicalInfo) {
49     SQLiteDatabase db = this.getWritableDatabase();
50     ContentValues contentValues = new ContentValues();
51     contentValues.put("name", name);
52     contentValues.put("breed", breed);
53     contentValues.put("age", age);
54     contentValues.put("weight", weight);
55     contentValues.put("medicalInfo", medicalInfo);
56     Cursor cursor = db.rawQuery("SELECT * FROM PetInfo WHERE name = ?", new String[] {name});
57     if(cursor.getCount() > 0) {
58         long result = db.update(tableName, contentValues, "name=?", new String[] {name});
59         if(result == -1) {
60             return false;
61         } else {
62             return true;
63         }
64     } else {
65         return false;
66     }
67 }
Run app
```

```
1 usage
85 void deleteAllData(){
86     SQLiteDatabase db = this.getWritableDatabase();
87     db.execSQL("DELETE FROM PetInfo");
88 }
89
90 1 usage
91 public Cursor getdata(){
92     SQLiteDatabase db = this.getWritableDatabase();
93     Cursor cursor = db.rawQuery("SELECT * FROM PetInfo", selectionArgs: null);
94     return cursor;
95 }
96
97 public String getName() {
98     SQLiteDatabase db = this.getWritableDatabase();
99     Cursor cursor = db.rawQuery("select * from PetInfo", selectionArgs: null);
100     if(cursor.moveToNext()){
101         return cursor.getString(columnIndex: 0);
102     }
103     return "name";
104 }
105 }
106
```

SharedPreferences for keep optional info

```
public class OptionInfo extends AppCompatActivity {

    2 usages
    TextView optionalInfo, food ,health,bath;
    5 usages
    SharedPreferences sp;
    2 usages
    Button btnBack;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_option_info);
        optionalInfo = findViewById(R.id.optionalInfo) ;
        food = findViewById(R.id.food);
        health = findViewById(R.id.health);
        bath = findViewById(R.id.bath);
        sp = getApplicationContext().getSharedPreferences( name: "UserPet", Context.MODE_PRIVATE);
        String optionalOption = sp.getString( key: "optionalInfo", defValue: "Nothing here...");
        optionalInfo.setText(optionalOption);
        food.setText(sp.getString( key: "food", defValue: "No preference at the moment"));
        health.setText(sp.getString( key: "health", defValue: "No vaccination planned"));
        bath.setText(sp.getString( key: "bath", defValue: "No bath planned"));
        btnBack = findViewById(R.id.btnBack);
        btnBack.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent( packageContext: OptionInfo.this, HomePage.class);
                startActivity(intent);
            }
        });
    }
}
```

Menu

```
@Override
public boolean onCreateOptionsMenu(Menu menu){
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.home_menu, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
    if(item.getItemId() == R.id.disconnect){
        Intent intent = new Intent( packageContext: HomePage.this, LoginActivity.class);
        startActivity(intent);
        return true;
    }else if (item.getItemId() == R.id.delete){
        db = new DBHelper( context: this);
        db.deleteAllData();
        petInfo = new PetInfo( context: this);
        petInfo.deleteAllData();
        sp = getSharedPreferences( name: "UserPet", Context.MODE_PRIVATE);
        sp.edit().clear().commit();
        Toast.makeText( context: this, text: "Your information is deleted", Toast.LENGTH_SHORT).show();
        Intent intent = new Intent( packageContext: HomePage.this, SignUpActivity.class);
        startActivity(intent);
        return true;
    }else if(item.getItemId()==R.id.info){
        Intent intent = new Intent( packageContext: HomePage.this, OptionInfo.class);
        startActivity(intent);
    }
    return super.onOptionsItemSelected(item);
}
```

1 usage

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/picture"
        android:title="Change your profile picture"/>
    <item android:id="@+id/info"
        android:title="See my information"/>
    <item android:id="@+id/delete"
        android:title="Delete my information"/>
    <item android:id="@+id/disconnect"
        android:title="Disconnect"/>
</menu>
```